

DSA STUDY BLUEPRINT SERIES

33.4 Sum Problem Optimal Approach

Finding Unique Quadruplets summing up to Target

Section 05 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Extending 3-Sum:** Pin two index loops (i and j) and run two-pointer sweeps for the remaining range.
- Prune checks to skip unnecessary evaluations.

Memory & Execution Blueprint

4-Sum nested anchors:

nums[i] • nums[j] • [Left Pointer → ⇐ Right
Pointer]

C++ Implementation Reference

Standard code layout and syntax

```
// 4-Sum: Nested loops for i and j, two pointers for left and right  
// Skip duplicates for i, j, left, and right indices to ensure unique results.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N^3)$
Space Complexity	$O(1)$ auxiliary

Key Practices & Gotchas

- **Important:** Use long long calculations to prevent integer overflow when target is large.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace check complement lookups in a hash map for `Two Sum` on `[2, 7, 11]` (target = 9):

Element Scan	Complement: <code>target - x</code>	Hash Map Search	Map Insertion
2	$9 - 2 = 7$	Not found	Insert 2 mapping index 0
7	$9 - 7 = 2$	Found 2 (at index 0!)	Return indices [0, 1]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Prefix Sum Math:** Resolve sub-array queries using math transformations. Complement hash lookups count occurrences instantly.
- **Hash Collisions:** Handled via separate chaining (lists link) or open addressing (linear index probing).
- **Related LeetCode Problems:** #560 Subarray Sum Equals K, #1 Two Sum, #49 Group Anagrams.